

Group 90 | MS3 Report

Vision:

Our vision has evolved from the conceptual planning in MS2 to a fully functional interactive N-body gravitational physics simulation. While MS2 outlined ideas like "creative mode" and parameter manipulation, MS3 represents a refinement of the scope based on what we would realistically implement. We focused on building a robust physics sandbox with pre-configured scenarios rather than free-form body placement. Our final system features five distinct scenarios (Three-Body Problem, Randomized 3-Body, Binary Star, Solar System, and Collision Course), with real-time 3D visualization with camera controls, interactive parameter editing through a sidebar interface, collision detection with an animation, and an adjustable simulation speed. The evolution from MS2 to MS3 has been converting the GUI from 2D to 3D, and adding depth over breadth, so finalizing our scope and ensuring what we ended up implementing was well polished and created a nice experience for the user.

Summary Of Progress:

Between MS2 and MS3, we improved upon our architecture and achieved our goal of creating a full 3D interactive physics simulation. We switched from OCaml's basic Graphics library to Raylib, due to its native support for 3D rendering, and adding camera controls (panning, tilting, and zooming) added a nice procedural starbox to help the user have something to reference the movement of the planets in the background. We expanded from a simple 2-body demonstration to 5 pre-configured scenarios showcasing different gravitational phenomena: binary star systems, solar systems, collision trajectories, and chaotic three-body problems, including a randomized one. We added visual features, including a trailing path of the planet's trajectory that fades over time. We also added collision detection and resolving in our physics engine, where if two planets collide, we simulate a perfectly inelastic collision and they "merge" together, forming a new planet that is made with the combined parameters of the two colliding planets (mass, density, color, radius). We also added a nice explosion animation in the front-end when a collision occurs. We also added nice controls in the frontend, such as if the user clicks on a planet a sidebar pops up where the user can adjust the planet's radius and density on sliders. We further refined the pausing, speeding up, and slowing down of the physics engine (we discovered a bug in our MS2 implementation).

Activity breakdown:

Felix Grimm (fjg45)

- **Responsibilities:** Backend physics engine development, Features ideator
- **Activities Participated In:**
 - Core engine architecture design
 - Physics algorithm implementation
 - Module interface design and documentation
 - Camera controls
 - Tuning of preset scenarios
 - MS3 report
- **Features Delivered:**
 - 3D vector mathematics library (vec3.ml/mli) with vector arithmetic, dot product, and normalization
 - Celestial body representation module (body.ml/mli) with mutable update functions
 - Collision detection and resolution in the physics engine
 - Physics engine (engine.ml/mli) including:
 - Gravitational force computation using Newton's law of universal gravitation
 - Net force accumulation across multiple interacting bodies
 - Numerical integration using the Euler method for time-stepping
 - mutable, functional world state updates
 - GUI:
 - Implemented camera panning, fixed render distance bug
 - Implemented procedural starbox
- **Hours Spent:** 10-12 hrs

Nathnael Tesfaw (nbt26)

- **Responsibilities:** Frontend 3D GUI development, visualization, and UI coordination
- **Activities Participated In:**
 - 3D graphics system migration from 2D Graphics library to Raylib
 - Frontend architecture restructuring and modularization
 - Rendering pipeline design and implementation
 - UI/UX coordination across all frontend components
 - Interactive controls and user input handling
- **Features Delivered:**
 - Complete 3D rendering engine using Raylib with perspective camera

- Frontend code organization: modular split of main.ml into simulation.ml, ui.ml, render.ml, cameracontrol.ml
- 3D camera system with orbital rotation, zoom, and spherical coordinate controls
- Collapsible sidebar UI with interactive sliders for real-time planet parameter editing (density/radius)
- Planet selection via 3D ray-casting and mouse clicking
- Collision warning displays, pause/speed controls (Z/X/P keys), and simulation status UI
- 60 FPS frame timing and main simulation loop coordination
- Ensure 80% coverage on bisect
- **Hours Spent:** 12 - 13 hrs

Anthony Paredes-Bautista (ap2357)

- **Responsibilities:** Scenario design, debugging, code documentation and project coordination
- **Activities Participated In:**
 - Scenario development and testing
 - GUI debugging and bug fixes
 - Code documentation and commenting
 - Project coordination and submission preparation
- **Features Delivered:**
 - Five pre-configured scenarios: Three-Body Problem, Randomized 3-Body, Binary Star, Solar System, Collision Course
 - Scenario module with orbital mechanics calculations (center-of-mass, circular orbits, collision trajectories)
 - GUI debugging and issue resolution across UI components
 - Code documentation and inline comments for all modules
 - MS3 submission materials (RepoURL.txt, gallery.yaml, MS3Report.pdf)
- **Hours Spent:** 10 hrs

Maliq Barnes (db927)

Responsibilities: Testing and quality assurance

Activities Participated In:

- Test suite development
- Test case design for physics modules
- Code validation and debugging

Features Delivered:

- Test cases for Vec3 module (vector operations, normalization, edge cases)
- Test cases for Body module (construction, accessors, immutable updates)
- Test cases for Engine module (gravitational force calculations, net force accumulation, time-stepping)
- Test cases for Scenario module (pre-configured celestial body systems with Five scenarios)
- Test cases for Simulation_state module (state management, trail tracking, collision animations)
- Test cases for Physics_system module (physics updates with substeps, collision detection, trail fading)
- OUnit2 test framework setup (test/dune, test/test_group90.ml)

Hours Spent: 3 hrs

Productivity analysis:

MS2

As a team, we were reasonably productive and accomplished the core objectives outlined in our sprints. We successfully delivered both the physics engine and GUI as planned, meeting our MS2 goals of a working simulation with user controls. However, our time estimates were somewhat inaccurate; we underestimated the complexity of the GUI implementation relative to the physics engine. The backend physics modules came together more quickly than anticipated, as the mathematical concepts translated smoothly into code. In contrast, integrating the Graphics library, implementing the unit-scaling system to bridge physics and pixel coordinates, and creating smooth user interactions took more time than we initially allocated. Despite this imbalance in effort distribution, we completed our sprint commitments and delivered a functional product. Moving forward to MS3, we now have a better understanding of the relative complexity of frontend versus backend work, which should help us create more accurate estimates for features like creative mode, additional user controls, and N-body expansion.

MS3

For MS3, most of our group was productive, well communicative, and contributed fairly to the final project. We successfully implemented the features we wanted to, and planned several “sprints” throughout the final week, and most of the group attended said sprints. We have several reach goals in mind if we had the time to implement them, and were actually able to implement one of them (popup sidebar). Some reach features were unfortunately not feasible in our timeframe, such as a spawn planet button or camera tracking of planets. So we were pretty accurate on our productivity estimate, if even conservative.